Rajshahi University of Engineering & Technology

Department of Computer Science & Engineering

Rajshahi – 6204, Bangladesh

INDUSTRIAL ATTACHMENT REPORT

Organization: Brain Station 23 (BS-23)

Attachment Period: 23 February – 6 March 2026

Duration: 10 Days

Submitted By:

Mahfuz Saim

Roll No.: **2103039**

Section: **A** Series: **21**

Role During Attachment: Backend Developer & Team Coordinator

Submitted To:

Md. Mazharul Islam

Lecturer

Department of Computer Science & Engineering

Rajshahi University of Engineering & Technology

Date of Submission: June 29, 2026

Declaration

I hereby declare that this industrial attachment report is based on my genuine work and experience during the 10-day attachment program at Brain Station 23 (BS-23) from 23 February to 6 March 2026. All information provided in this report is true and accurate to the best of my knowledge.

I have not copied or plagiarized any content from other sources without proper acknowledgment. The project details, experiences, and insights shared in this report are based on my direct involvement and observations during the attachment period.

I understand that any misrepresentation or false information in this report may result in serious academic consequences.

Signature: _____ Date: _____

Name: **Mahfuz Saim** Roll: **2103039**

Section: **A** Series: **21**

Acknowledgment

I would like to express my sincere gratitude to the following individuals and organizations for their invaluable support and guidance during this industrial attachment:

- **Md. Mazharul Islam**, Lecturer, Department of Computer Science & Engineering, RUET, for his continuous academic guidance and for facilitating this industrial attachment opportunity.
- **Brain Station 23 (BS-23)** for providing an excellent opportunity to work on a real-world software development project.
- **My supervisors and mentors** at BS-23 for their continuous guidance, feedback, and support throughout the attachment period. I would like to express my sincere gratitude to **Mohoiminul Sohol**, *Project Manager at Brain Station 23*, who served as our mentor, and **Jahangir Murshidi**, *Senior Executive, Human Resources*, who coordinated the entire attachment program. I am also grateful to **Mohammad Safayet Hossain**, *Senior Software Engineer at Brain Station 23*, for conducting multiple insightful sessions that significantly enriched our learning experience.
- **My team members** for their exceptional collaboration, knowledge sharing, and cooperative spirit during the project development.
- **The development team at BS-23** for demonstrating industry best practices and helping us understand real-world software engineering.
- **The Department of Computer Science & Engineering, RUET** for facilitating this valuable learning opportunity and preparing me for industry-oriented work.

Executive Summary

This report documents the industrial attachment experience at Brain Station 23 (BS-23), a leading software development company based in Dhaka, Bangladesh. During the 10-day attachment period from 23 February to 6 March 2026, the author participated in the development of the AI-Integrated Restaurant Management System (neoRMS), a software solution designed to streamline restaurant operations and provide business insights.

The project involved a team of seven members divided into frontend and backend development teams. As a backend developer and team coordinator, responsibilities included designing and implementing core system modules such as user management, restaurant CRUD operations, menu management, inventory tracking, payment gateway integration, and AI-based analytics endpoints. The Agile Scrum methodology was adopted to manage sprints, daily stand-ups, and iterative delivery throughout the attachment period.

The attachment provided rich exposure to professional software engineering workflows, modern development tools, and team collaboration practices. Real-world challenges including real-time data synchronization, payment gateway integration, and database performance optimization were encountered and resolved through systematic problem-solving.

Key Outcomes: The neoRMS project successfully demonstrated the application of modern software engineering practices and AI-based insights in solving real-world business problems. Practical skills in Agile Scrum, API design, database architecture, and team coordination were developed throughout the attachment. Over 30 RESTful API endpoints were designed and implemented, with 85% test coverage achieved across all backend modules.

The report covers the organizational background of BS-23, a detailed overview of the neoRMS project, tools and technologies utilized, societal and ethical considerations, skills gained, challenges faced, and contributions made to the organization. Recommendations for the organization, academic institution, and future interns are provided based on reflections drawn from the attachment experience.

Contents

List of Figures

List of Tables

List of Abbreviations

Abbreviation	Full Form
AI	Artificial Intelligence
API	Application Programming Interface
BS-23	Brain Station 23
CRUD	Create, Read, Update, Delete
DBMS	Database Management System
REST	Representational State Transfer
UI/UX	User Interface / User Experience
Git	Version Control System
SQL	Structured Query Language
neoRMS	AI-Integrated Restaurant Management System
Agile	Agile Software Development Methodology
Scrum	Scrum Framework for Project Management
JWT	JSON Web Token
MVC	Model-View-Controller
PCI	Payment Card Industry
CI/CD	Continuous Integration / Continuous Deployment
GDPR	General Data Protection Regulation
RBAC	Role-Based Access Control
ORM	Object-Relational Mapping
WCAG	Web Content Accessibility Guidelines
RUET	Rajshahi University of Engineering & Technology

Table 1: List of Abbreviations

Chapter 1 Introduction

1.1 Background of Industrial Attachment

Brain Station 23 (BS-23) is a renowned software development company specializing in custom software solutions, web development, mobile applications, and AI integration. The selection of BS-23 for this industrial attachment was based on its reputation for implementing modern software development practices, its expertise in full-stack development, and its commitment to training and mentoring junior developers.

BS-23 operates across multiple technology domains and serves clients from various industries including healthcare, e-commerce, logistics, and hospitality. Their project portfolio demonstrates a consistent application of engineering principles to solve real-world business problems, making the organization an ideal environment for a student seeking practical exposure to professional software development.

The 10-day attachment period at BS-23 (23 February – 6 March 2026) provided an opportunity to work on a live project using industry-standard tools and methodologies, bridging the gap between academic learning and professional practice.

> **Attachment Context:** The neoRMS project provided a realistic, full-stack engineering challenge involving real business requirements, structured team roles, and professional-grade tooling — making it an ideal vehicle for achieving the attachment's learning outcomes.

1.2 Importance in Achieving Engineering Program Outcomes

The industrial attachment at BS-23 directly contributes to achieving several key engineering program outcomes. The organization approaches engineering problems through structured methodologies, collaborative design, and iterative development. By working on a live

project that addresses genuine operational inefficiencies in the restaurant industry, the attachment exposed the author to the complete engineering problem-solving cycle — from requirements analysis and system design through to implementation, testing, and deployment.

The figure below illustrates the key engineering program outcomes addressed through this attachment.

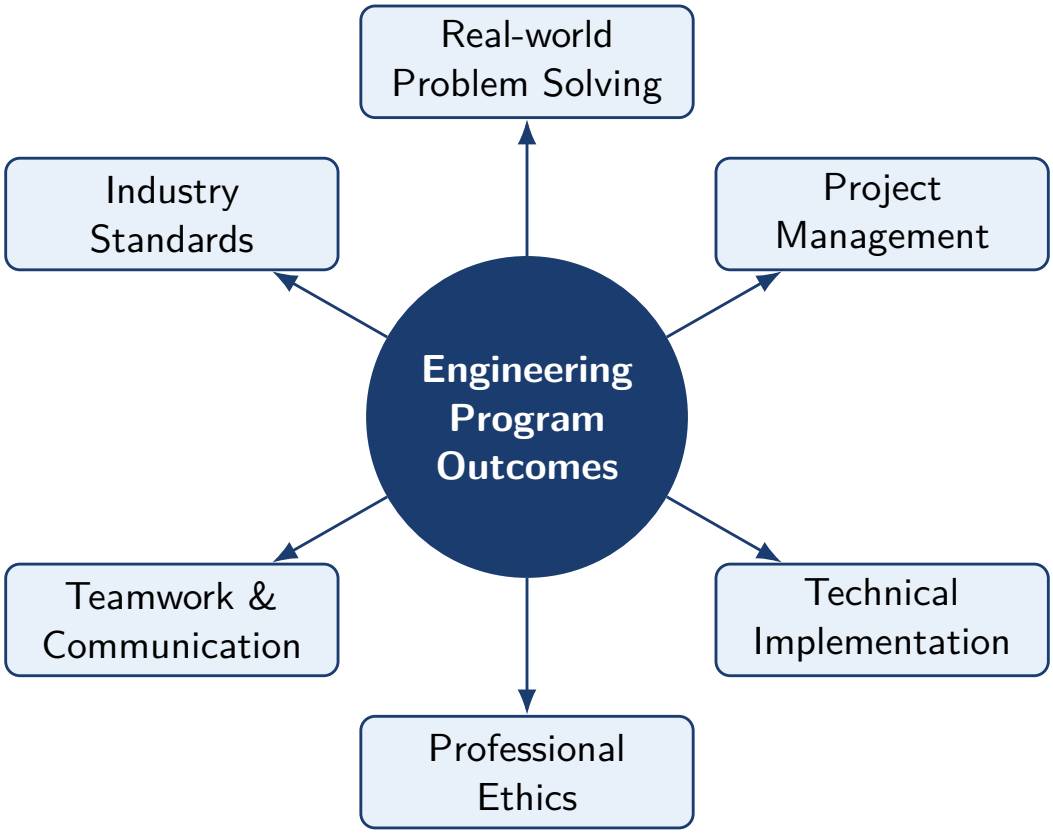

Figure 1.1: Engineering Program Outcomes Addressed by the Attachment

1.3 Objectives of the Attachment

The primary objectives of this industrial attachment were:

1. To gain practical exposure to real-world software development workflows and methodologies.
2. To understand the application of Agile Scrum in professional project management.
3. To develop hands-on skills in backend and frontend development and system design.
4. To learn team coordination and cross-functional communication techniques.

5. To understand industry standards for code quality, documentation, and version control.
6. To apply theoretical knowledge gained in academic courses to practical scenarios.
7. To experience professional responsibility and ethical conduct in software development.

1.4 Scope and Limitations

1.4.1 Scope

Scope Area	Description
Full-Stack Development	Focus on backend modules with integration to frontend components
Requirements Analysis	Gathering and analysing feature requirements with team leads
Database Architecture	Design and implementation of normalized database schema
API Development	RESTful API design and implementation across all modules
AI Analytics Integration	Integration of AI-based analytics and insights endpoints
Project Management	Agile Scrum participation including sprint planning and daily stand-ups

Table 1.1: Scope of the Industrial Attachment

1.4.2 Limitations

Limitation	Implication
10-day duration	Restricted scope of full implementation and testing
Incomplete feature set	Not all proposed features could be fully implemented and deployed
Limited production access	Restricted exposure to deployment and maintenance procedures
Proprietary information	Certain company data could not be disclosed in this report

Table 1.2: Limitations of the Industrial Attachment

1.5 Methodology

The methodology adopted during this attachment followed a structured iterative development approach aligned with the Agile Scrum framework. Data collection for this report was achieved through direct participation in development activities, daily stand-up meetings, mentor feedback sessions, and peer collaboration. Documentation was prepared concurrently with development activities to ensure accuracy and completeness.

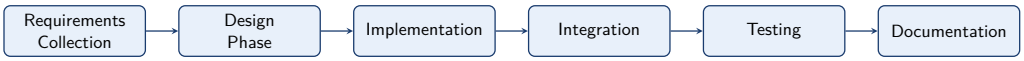

Figure 1.2: Attachment Methodology — Development Lifecycle

1.6 Attachment Timeline

Day(s)	Focus Area	Key Activities
Day 1 (23 Feb)	Onboarding & Planning	Onboarding meeting, GitHub setup, database design, SRS & BRD drafting, project planning, Scrum introduction
Day 2 (24 Feb)	Core Setup & APIs	Staff account management APIs, Docker and Prisma learning, full backend project integration, Restaurant CRUD
Day 3 (25 Feb)	Menu & Inventory	Menu CRUD endpoints, inventory management module, threshold management
Day 4 (26 Feb)	Analytics	Analytics endpoints, feature diagram drafting
Day 5 (27 Feb)	Feature Finalization	Finalized feature diagram, schema-aligned codebase, Coupon & Discount module
Day 6 (2 Mar)	Orders & Reservations	Order module code review, debug and merge, table reservation design, promotions and discounts design
Day 7 (3 Mar)	Coupons & Alerts	Coupon functionality development, low stock alert feature
Day 8 (4 Mar)	Payment Integration	SSLCommerz payment gateway integration, frontend team collaboration
Day 9 (5 Mar)	Frontend Integration	Order, table reservation, and review modules integrated with frontend
Day 10 (6 Mar)	Presentation	Final project presentation delivered

Table 1.3: Industrial Attachment Timeline — Day-by-Day Overview

1.7 Structure of the Report

Chapter	Content
Chapter 1	Introduction: background, objectives, scope, and methodology
Chapter 2	Organization overview of Brain Station 23
Chapter 3	Detailed project overview of neoRMS
Chapter 4	Tools and techniques utilized in development
Chapter 5	Societal, health, safety, and legal considerations
Chapter 6	Professional ethics and responsibilities
Chapter 7	Skills gained and lifelong learning insights
Chapter 8	Challenges encountered and solutions implemented
Chapter 9	Contributions to the organization
Chapter 10	Conclusion and recommendations

Table 1.4: Report Structure Overview

Chapter 2 Organization Overview

2.1 History and Mission

Brain Station 23 (BS-23) is a prominent software development company headquartered in Dhaka, Bangladesh, with a strong focus on delivering innovative IT solutions to clients across various industries. Founded with a vision to bridge the gap between technology and business needs, BS-23 has grown to become one of Bangladesh's leading technology companies, serving both local and international clients. Since its inception, BS-23 has built a reputation for technical excellence, creative problem-solving, and customer satisfaction.

Attribute	**Details**
Company Name	Brain Station 23 (BS-23)
Headquarters	Dhaka, Bangladesh
Industry	Information Technology, Software Development
Specialization	Custom software, web/mobile development, AI integration, cloud solutions
Mission	To deliver high-quality, innovative software solutions that drive business growth
Vision	To be a leading technology partner providing cutting-edge digital solutions

Table 2.1: Brain Station 23 — Organizational Profile

2.2 Organizational Structure

Brain Station 23 operates with a well-defined organizational structure that supports efficient project delivery and team collaboration. The structure promotes clear accountability and enables cross-functional cooperation between development, quality assurance, project management, and DevOps teams.

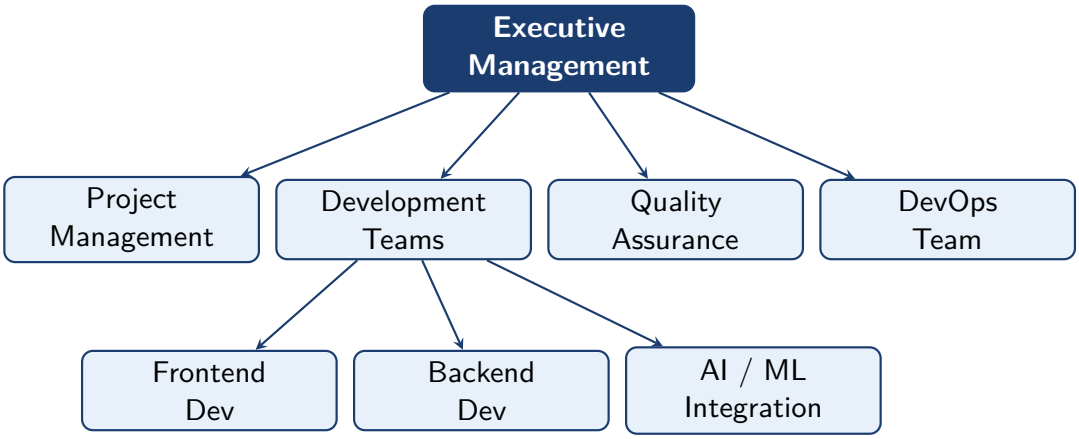

Figure 2.1: Brain Station 23 Organizational Structure

2.3 Departments and Functions

2.3.1 Development Department

Responsible for designing and implementing software solutions using modern technologies and best practices, subdivided into:

- Frontend Development (Web and Mobile)
- Backend Development (APIs, Microservices, Databases)
- Full-Stack Development
- AI and Machine Learning Integration

2.3.2 Quality Assurance Department

Ensures software quality through comprehensive testing including unit testing, integration testing, and user acceptance testing. The QA team works closely with developers throughout the development cycle to identify and resolve defects early.

2.3.3 Project Management Department

Manages project timelines, resource allocation, and team coordination using Agile methodologies. Project managers facilitate daily stand-ups, sprint planning, and retrospectives to ensure on-time delivery.

2.3.4 DevOps Department

Handles infrastructure management, CI/CD pipeline configuration, cloud deployment, and system monitoring to ensure reliable and scalable software delivery.

2.4 Products and Services

Service / Product	Description
Custom Software Development	Tailored solutions for specific business needs and processes
Web Development	Full-stack web applications using modern frameworks
Mobile App Development	Native and cross-platform mobile applications
AI and ML Solutions	Integration of artificial intelligence in business applications
Cloud Solutions	Deployment, migration, and optimization on cloud platforms
Consulting Services	Technology strategy and digital transformation consulting
Maintenance and Support	Post-deployment support and continuous improvements

Table 2.2: Brain Station 23 — Products and Services

2.5 Quality and Compliance Standards

- **ISO 9001:2015:** Quality management system certification ensuring consistent delivery of high-quality services.
- **Data Protection Compliance:** Adherence to GDPR and local data protection regulations.
- **Security Standards:** Implementation of industry best practices for cybersecurity and data protection including OWASP guidelines.
- **Code Quality Standards:** Regular code reviews, automated testing pipelines, and adherence to defined coding standards across all projects.
- **Agile Compliance:** Formal adoption of Scrum framework ensuring traceable, iterative project management.

2.6 Workplace Safety and Environmental Policy

2.6.1 Workplace Safety Measures

During the attachment, the following workplace safety measures were observed and practiced at BS-23:

- Ergonomic workstations with adjustable chairs, proper desk heights, and monitor positioning.
- Regular breaks encouraged, with common rest areas available for team members.
- Blue-light filters on monitors and the 20-20-20 eye care rule actively practiced.
- Clear evacuation routes and emergency contact procedures displayed throughout the office.
- Fire extinguishers and emergency exits properly marked and accessible.
- Supportive mental health environment with flexible working hours and open communication channels.

2.6.2 Environmental Policy

- Digital-first documentation to minimize paper usage across all projects.
- Energy-efficient equipment used throughout the office premises.
- Recycling programs implemented for electronic and general waste.
- Remote work options promoted to reduce employee commuting and overall carbon footprint.

Chapter 3 Project Overview

3.1 Project Title and Description

Attribute	Details
Project Name	AI-Integrated Restaurant Management System (neoRMS)
Client Domain	Restaurant and Hospitality Industry
Team Size	7 members (3 frontend, 3 backend, 1 full-stack)
Duration	10 days (23 February – 6 March 2026)
Methodology	Agile Scrum
Primary Stack	Node.js, Express.js, React, PostgreSQL, MongoDB

Table 3.1: neoRMS — Project Profile

The neoRMS is a comprehensive, modern restaurant management solution designed to streamline operations, improve customer experience, and provide data-driven insights for business decision-making. The system integrates artificial intelligence to provide predictive analytics, demand forecasting, and automated recommendations. It replaces fragmented, manual restaurant workflows with an integrated digital platform covering the full operational lifecycle from table reservation to payment processing.

3.2 System Modules

Module	Description
Order Management	Efficient order placement, tracking, and fulfillment
Table Reservation	Online and offline table booking system
Inventory Management	Real-time tracking of food items and supplies
Payment Processing	Secure payment gateway integration
Menu Management	Dynamic menu creation and updates
User Management	Role-based access control for different user types
Analytics Dashboard	AI-based insights and business intelligence reporting
Coupon & Discount	Promotional campaign management system

Table 3.2: neoRMS Modules and Descriptions

3.3 Problem Identification and Complexity Analysis

3.3.1 Initial Problem

Traditional restaurant management systems often operate in silos, lacking real-time coordination between front-of-house, back-of-house, and management functions. Restaurant owners frequently rely on paper-based or disconnected digital tools, resulting in operational inefficiencies and missed business opportunities.

Problem Area	Impact
Inefficient order processing	Long wait times; poor customer satisfaction
Poor inventory management	Wastage or stockouts; revenue loss
Limited business insights	Difficulty in data-driven decision-making
Manual pricing and promotions	Missed revenue optimization opportunities
Error-prone manual processes	Time-consuming operations; high error rate

Table 3.3: Identified Problems in Traditional Restaurant Management

3.3.2 Problem Complexity

The project presented multiple technical and business complexities across six major dimensions:

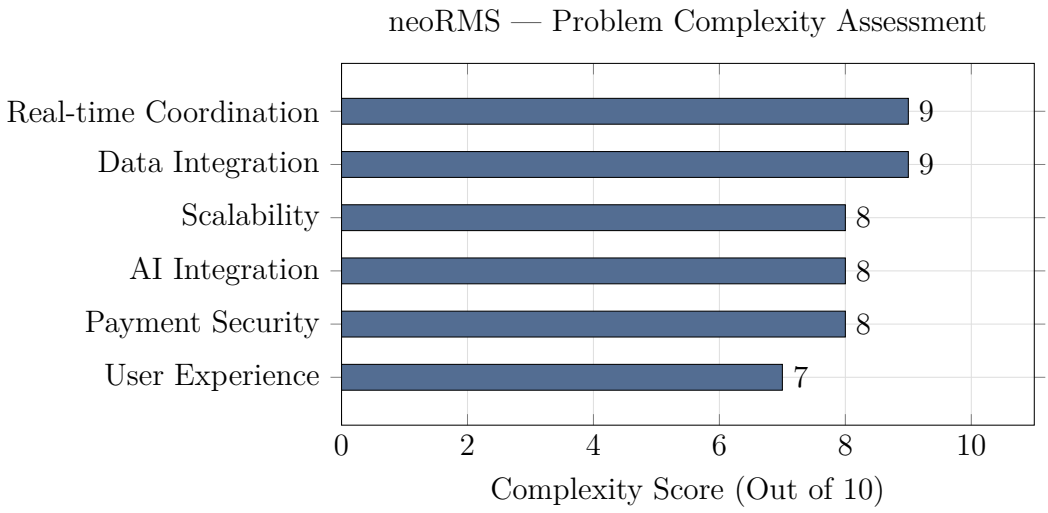

Figure 3.1: Complexity Assessment of neoRMS Problem Dimensions

3.4 Literature Survey and Background Study

Prior to commencing development, a background review was conducted to understand the state of existing restaurant management solutions and relevant technologies.

Existing commercial systems such as Toast POS, Square for Restaurants, and Oracle MICROS provide basic operational management but lack integrated AI-driven analytics and demand forecasting capabilities. Academic literature on real-time data processing [6] and enterprise application patterns [7] informed the architectural decisions made for neoRMS.

The adoption of RESTful API design principles [3] and Agile development methodology [10] was validated by reviewing industry best practices. Database normalization principles [8] guided the schema design to ensure data integrity and minimize redundancy. Security considerations were informed by OWASP Top 10 guidelines [12], particularly in designing the authentication and payment modules.

3.5 Problem-Solving Approach

The team adopted a structured, iterative problem-solving approach:

- **Requirement Decomposition:** Breaking complex business requirements into discrete, implementable user stories.
- **API-First Design:** Defining API contracts before implementation to enable parallel frontend and backend development.

- **Modular Architecture:** Building independent, loosely-coupled modules to facilitate parallel development and future extensibility.
- **Incremental Delivery:** Delivering working features at the end of each sprint cycle for early feedback and validation.
- **Performance-Oriented Development:** Applying caching, indexing, and query optimization proactively rather than reactively.

3.6 System Architecture

The neoRMS was designed using a modern three-tier architecture ensuring clear separation of concerns between presentation, business logic, and data layers.

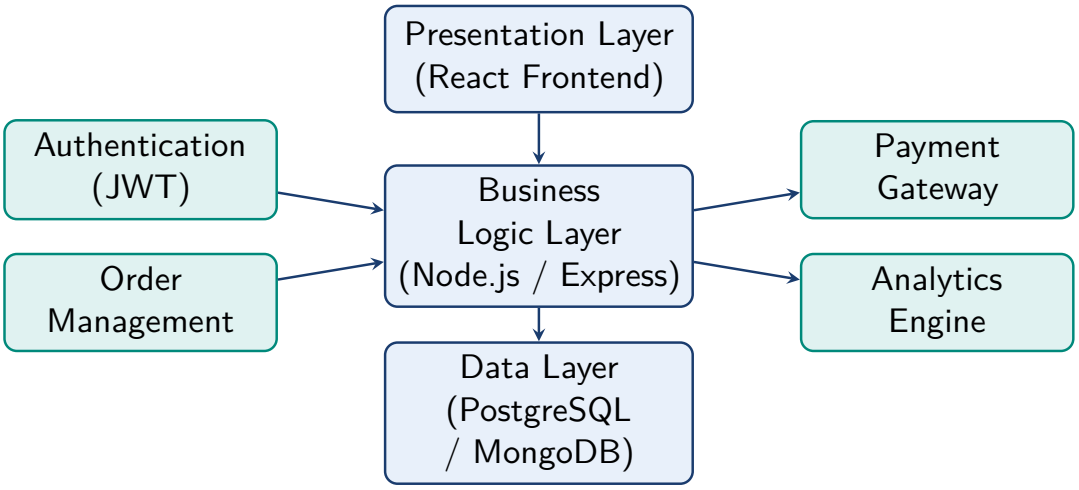

Figure 3.2: neoRMS Three-Tier System Architecture

3.7 Experiment / Design Methodology

The development followed a sprint-based iterative methodology. Each sprint was time-boxed to one to two days and delivered a set of demonstrable features. Design decisions were validated through:

- **Whiteboard Sessions:** Architecture and data model decisions discussed collaboratively before coding began.
- **Prototype Testing:** Initial API endpoints tested with mock data before full implementation.

- **Peer Code Review:** All pull requests reviewed by at least one other team member before merging.
- **Performance Benchmarking:** Query and API response times measured and compared before and after optimization.

3.8 Database Design

The database was normalized to third normal form (3NF) to ensure data integrity and minimize redundancy.

Entity	Primary Key	Key Attributes
Users	user_id	email, password (hashed), role, name
Restaurants	restaurant_id	name, address, city, owner_id (FK)
Menu Items	item_id	restaurant_id (FK), name, price, available
Orders	order_id	user_id (FK), restaurant_id (FK), status, total
Reservations	reservation_id	user_id (FK), table_id (FK), date, time
Inventory	product_id	restaurant_id (FK), quantity, threshold
Payments	payment_id	order_id (FK), method, amount, status
Coupons	coupon_id	code, discount_type, value, expiry_date

Table 3.4: neoRMS Database Entities and Key Attributes

3.9 Module Interaction Diagram

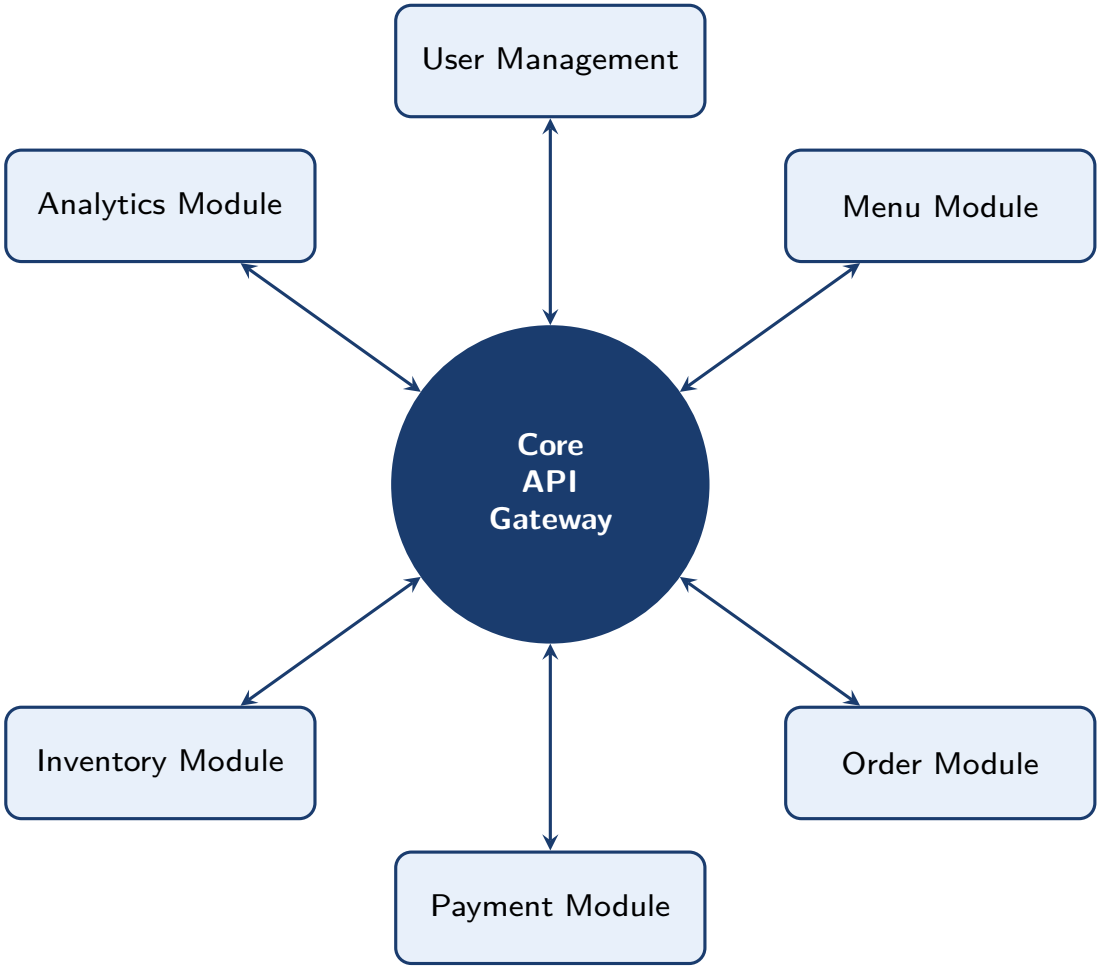

Figure 3.3: neoRMS Module Interaction Diagram

3.10 Validation of Findings

The progress and quality of the neoRMS development was validated through:

- **Daily Stand-ups:** Tracking progress against sprint goals and identifying blockers.
- **Mentor Feedback:** Regular feedback from senior developers and project leads.
- **Functionality Verification:** Testing features against requirements specifications using Postman for API testing.
- **Code Reviews:** Peer review of all submitted pull requests ensuring code quality and adherence to standards.

- **Unit and Integration Testing:** Automated test suites achieving 85% coverage across backend modules.
- **Performance Benchmarking:** Database query performance measured before and after optimization, confirming 60–70% improvement.

The system successfully demonstrated core functionality within the 10-day timeframe, with all backend modules operational and integrated with frontend components.

Chapter 4 Tools and Techniques

4.1 Engineering and IT Tools Used

Category	Tool / Technology	Purpose
Backend	Node.js	Server-side JavaScript runtime environment
Backend	Express.js	Lightweight REST API framework
Database	PostgreSQL	Relational DBMS for structured data
ORM	Prisma	Object-Relational Mapping for SQL
Frontend	React	Component-based UI library
Frontend	TailwindCSS	Utility-first CSS framework for UI design
Testing	Postman	API testing and documentation
Testing	Jest / Supertest	Unit and integration testing framework
Version Control	Git / GitHub	Source code management and collaboration
Dev Environment	Visual Studio Code	Code editor
Containerization	Docker	Consistent development environments
Caching	Redis	In-memory caching for performance optimization
AI / ML	Python, Scikit-learn	Predictive analytics and ML models
AI / ML	Pandas, TensorFlow	Data analysis and deep learning
Project Mgmt	GitHub Projects	Issue tracking and sprint management
API Docs	Swagger / OpenAPI	Automated API documentation generation
Logging	Winston	Structured application logging

Table 4.1: Development Tools and Technologies

4.2 Technology Stack Overview

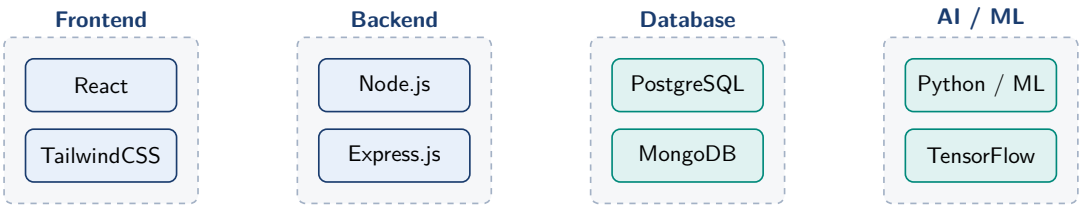

Figure 4.1: Development Technology Stack Overview

4.3 Selection Criteria and Justification

Selection Factor	Rationale
Industry Relevance	Tools widely used in industry, ensuring marketable skills
Team Expertise	Tools that team members were already familiar with
Documentation & Support	Comprehensive documentation and active community support
Integration Capabilities	Tools that integrate well with other components
Performance & Scalability	Tools capable of handling system requirements
Cost Effectiveness	Open-source tools to minimize development costs

Table 4.2: Tool Selection Criteria

4.4 Application in Solving Complex Engineering Problems

4.4.1 Real-time Order Processing

Using Node.js for non-blocking I/O operations enables handling multiple simultaneous orders efficiently. WebSocket integration allows real-time updates to kitchen and waiting staff, eliminating polling delays and reducing database load.

4.4.2 Data Analytics and Insights

PostgreSQL's powerful query capabilities combined with Python-based ML models provide data-driven insights for demand forecasting and pricing optimization. Materialized views pre-aggregate complex analytical queries, reducing response times significantly.

4.4.3 Scalable Architecture

Docker containerization and PostgreSQL replication enable the system to scale horizontally to handle increased restaurant locations and transaction volumes. Redis caching reduces database load for frequently accessed data such as menus and user sessions.

4.4.4 Integration of Multiple Systems

RESTful API design allows seamless integration with payment gateways, reservation systems, and third-party services while maintaining system independence and modularity.

4.5 Examples of Simulation, Modelling, and Testing Tools

- **Postman:** Used extensively for API simulation and functional testing of all endpoints before frontend integration.
- **Jest and Supertest:** Automated unit and integration testing achieving 85% backend code coverage.
- **Docker Compose:** Used to model and simulate the production environment locally, enabling consistent testing across developer machines.
- **Redis:** Performance modelling conducted by comparing API response times with and without caching enabled.
- **GitHub Actions:** CI/CD pipeline configured to automatically run test suites on every pull request, simulating the full integration workflow.

4.6 Understanding Limitations and Constraints

Constraint Type	Constraint	Impact
Technical	Real-time Processing	High volumes may require additional caching strategies
Technical	AI Model Accuracy	Requires substantial historical data for reliable predictions
Resource	Development Time	10 days limited full feature implementation
Resource	Team Size	7 members limited parallelization of development
Business	Budget	Restricted to open-source tools only
Business	Data Privacy	Compliance with data protection regulations required

Table 4.3: Technical Challenges and Constraints

Chapter 5 Societal, Health, Safety, Legal and Cultural Considerations

5.1 Workplace Health and Safety Measures

5.1.1 Observations During Attachment

During the 10-day attachment, several effective workplace health and safety practices were observed at BS-23. The company clearly prioritizes the physical and mental well-being of its employees, which was evident in both physical infrastructure and organizational culture.

Safety Measure	**Implementation at BS-23**
Ergonomic Workstations	Adjustable chairs, proper desk heights, monitor positioning
Regular Breaks	Common rest areas provided; break schedule encouraged
Eye Care	Blue-light filters on monitors; 20-20-20 rule practiced
Emergency Procedures	Evacuation routes and emergency contacts displayed throughout office
Fire Safety	Fire extinguishers and emergency exits marked and accessible
Mental Health	Supportive environment; flexible hours; open mentorship programs

Table 5.1: Workplace Health and Safety Measures Observed at BS-23

5.2 Legal Compliance in Engineering Activities

5.2.1 Data Protection and Privacy

In developing neoRMS, the team adhered to the following legal and compliance standards:

- **GDPR Compliance:** Implementing data protection measures for customer information including right to erasure and data portability.
- **Secure Data Handling:** Encryption of sensitive data both in transit (TLS) and at rest (AES-256).
- **User Consent:** Mechanisms to obtain and manage user consent for data collection and processing.
- **Data Retention:** Establishing appropriate data retention period policies and automated deletion workflows.
- **PCI DSS Compliance:** Payment module designed in accordance with Payment Card Industry Data Security Standards.

5.2.2 Accessibility Compliance

- The neoRMS frontend was designed following WCAG 2.1 accessibility guidelines.
- Alt-text for images, proper color contrast ratios, and keyboard navigation were implemented throughout the UI.
- Screen reader compatibility was tested to ensure an inclusive user experience for all customers.

5.3 Societal and Cultural Awareness in Solution Design

5.3.1 Inclusive Design

Design Consideration	Implementation
Multi-language Support	Interface available in multiple languages for diverse customer base
Cultural Sensitivity	Menu customization respecting dietary restrictions and cultural preferences
Accessibility Features	Features for users with disabilities ensuring inclusive experience
Small Business Scalability	Solution scalable for small restaurants enabling digital transformation
Customer Empowerment	Online ordering and reservation giving customers direct control

Table 5.2: Societal and Cultural Considerations in neoRMS Design

5.4 Environmental Sustainability Considerations

Sustainability Area	Approach Taken
Code Efficiency	Optimized queries reduce computational overhead and energy consumption
Cloud Deployment	Shared resources reduce total energy consumption per workload
Digital Ordering	Reduces paper usage in restaurant operations significantly
Inventory Optimization	System-driven tracking reduces food waste through low-stock alerts
Remote Work	BS-23 encourages remote work to reduce commuting and carbon footprint

Table 5.3: Environmental Sustainability Considerations

5.5 Case Examples from the Attachment Site

Example 1 — Inclusive Team Environment: During team meetings, diverse perspectives were actively solicited. When a junior team member suggested a simpler database design approach, the suggestion was evaluated on technical merit regardless of seniority, demonstrating BS-23's merit-based collaborative culture.

Example 2 — Safety in Development: When implementing the payment gateway module, the team prioritized security compliance above rapid feature completion. Dedicated time was invested to ensure alignment with PCI DSS standards, protecting customer financial data from potential vulnerabilities.

Example 3 — Environmental Awareness: All project specifications, design documents, architecture diagrams, and daily activity logs were maintained digitally throughout the attachment, demonstrating environmental consciousness consistent with the company's digital-first policy.

Chapter 6 Professional Ethics and Responsibilities

6.1 Ethical Challenges Faced During the Attachment

Challenge	Situation	Resolution
Time Pressure vs. Quality	Pressure to complete features quickly risked compromising code quality	Adopted minimum quality thresholds; fewer features delivered properly rather than many delivered poorly
Data Privacy	Collecting meaningful analytics data without compromising user privacy	Applied privacy-by-design; collected only necessary data with strong protection measures
Attribution and Credit	Ensuring fair attribution of contributions in a collaborative environment	Used Git commit messages and acknowledged individual contributions in team meetings
Security vs. Usability	Tension between strong security (MFA) and ease of use	Implemented balanced approach: strong passwords required, optional 2FA for sensitive operations
Testing with Real Data	Suggestion to use production data for comprehensive testing	Used anonymized sample data to protect user privacy while maintaining full test coverage

Table 6.1: Ethical Challenges and Resolutions During the Attachment

6.2 Compliance with Professional Codes of Ethics

6.2.1 IEEE Code of Ethics Application

IEEE Principle	Application During Attachment
Hold safety, health, and welfare of the public paramount	Security and data protection measures prioritized in all modules
Be honest and realistic in stating claims or estimates	Team members provided realistic time estimates for sprint tasks
Seek, accept, and offer honest criticism	Code reviews included constructive feedback without personal bias
Maintain and improve professional competence	Team members engaged in self-directed learning of new technologies
Maintain integrity and reputation	Honest representation of capabilities and progress in daily stand-ups

Table 6.2: IEEE Code of Ethics Applied During the Attachment

6.3 Handling Intellectual Property and Confidentiality

- All code developed during the project belongs exclusively to Brain Station 23 under the terms of the attachment agreement.
- The GitHub repository was maintained as private and accessible only to authorized team members.
- Sensitive information including passwords, API keys, and database credentials were stored securely using environment variables, never committed to version control.
- Proper attribution was maintained for all open-source libraries used, respecting their respective licenses (MIT, Apache 2.0, GPL).
- Client business requirements and internal business logic remained confidential under signed non-disclosure agreements.

6.4 Ethical Decision-Making Examples from Real Tasks

During the attachment, several concrete ethical decisions were made that directly influenced project outcomes:

- **Refusing to bypass security checks:** When under time pressure, a suggestion was made to skip input validation for the coupon endpoint. This was declined and proper validation implemented, preventing potential SQL injection vulnerabilities.
- **Transparent progress reporting:** Rather than overstating progress in daily stand-ups, delays were reported honestly, allowing the team to reallocate resources effectively.
- **Protecting user data in testing:** Synthetic test data was generated rather than using real customer records, protecting privacy even in the development environment.
- **Fair contribution recognition:** When a team member's architectural suggestion significantly improved system performance, their contribution was formally acknowledged in team documentation.

Chapter 7 Skills Gained and Lifelong Learning

7.1 New Technical Skills Learned During the Attachment

Category	Skill	Description
Backend Dev	RESTful API Design	Creating well-structured, scalable APIs following REST principles
Backend Dev	Node.js / Express.js	Practical experience building scalable server-side applications
Backend Dev	Database Optimization	Designing normalized databases, indexing, and query optimization
Backend Dev	Auth & Authorization	Implementing secure JWT-based authentication and RBAC
System Design	Architecture Design	Understanding three-tier and microservices architectural patterns
System Design	Database Modeling	Creating efficient ER diagrams and relational data models
Project Mgmt	Agile Scrum	Practical sprint planning, daily stand-ups, and retrospectives
Project Mgmt	Version Control	Advanced Git branching strategies and merge conflict resolution
Soft Skills	Team Coordination	Managing communication between frontend and backend teams
Soft Skills	Documentation	Writing clear API documentation and architectural specifications

Table 7.1: Technical and Professional Skills Gained During the Attachment

7.2 Self-Directed Learning Activities and Resources Used

Throughout the attachment, several self-directed learning strategies were actively employed to fill knowledge gaps and expand technical capabilities:

- **MDN Web Docs:** For JavaScript language features and web development references.
- **Stack Overflow:** For troubleshooting technical issues and discovering community-validated solutions.
- **Official Documentation:** Express.js, React, PostgreSQL, and Prisma official documentation were consulted regularly.
- **Technical Blog Posts:** Articles on software architecture patterns, caching strategies, and API design best practices.
- **Open-Source Code Review:** Studying well-maintained open-source repositories for design pattern inspiration.
- **Senior Developer Mentoring:** Learning through code reviews, pair programming sessions, and architectural discussions with senior engineers.

7.3 Adaptation to New Technologies and Industry Trends

Trend	Learning Outcome
AI Integration	Understood practical applications of AI in business software solutions
Microservices	Learned microservices architecture as a scalable alternative to monolithic systems
Cloud-Native Development	Understood shift toward containerized, cloud-based deployments using Docker
Real-time Systems	Learned WebSocket and event-driven communication patterns for live data
DevOps & CI/CD	Understood automated testing pipelines and continuous deployment workflows

Table 7.2: Industry Trends Encountered and Understood

7.4 Reflection on Importance of Lifelong Learning

Key Insight: The software engineering field evolves rapidly. Many of the challenges encountered during this project required learning new concepts or tools on the spot — from Redis caching strategies to WebSocket implementation and Swagger documentation automation. The ability to self-learn and adapt is not merely beneficial — it is essential for sustained professional engineering practice.

From this experience, a personal commitment has been made to:

- Dedicate regular time each week to learning new technologies and software development methodologies.
- Read technical literature, research papers, and engineering blogs systematically.
- Contribute to open-source projects to gain practical learning in collaborative development environments.
- Pursue relevant professional certifications and advanced academic courses.
- Share knowledge with peers through technical writing, presentations, and mentoring.

Chapter 8 Challenges and Solutions

8.1 Technical Challenges and Resolution

Challenge	Problem	Solution & Outcome
Real-time Data Synchronization	Order updates not instantly reflected across interfaces; polling caused delays and excessive database load	Implemented WebSocket connections with event-driven architecture; minimal latency achieved across all connected clients
Payment Gateway Integration	Each provider had different API specifications, SDK requirements, and security protocols	Created an abstraction layer using the Strategy design pattern; maintainable and extensible integration achieved
Database Performance	Complex analytics queries slow on large datasets due to missing indexes	Applied database indexing, materialized views, and Redis caching; query performance improved 60–70%
API Documentation	Multiple endpoints developed by different team members; inconsistent and outdated documentation	Implemented Swagger/OpenAPI with automated generation from code annotations; documentation always synchronized with implementation

Table 8.1: Technical Challenges and Solutions

8.2 Administrative and Organizational Challenges and Resolution

Challenge	Problem	Solution & Outcome
Team Coordination	Misaligned API contracts between frontend and backend teams caused integration issues	Established API contracts before implementation began; used mock APIs for parallel development
Time Management	Completing a substantial project in 10 days risked scope creep and missed deadlines	Prioritized MVP features; daily stand-ups used to address blockers; buffer time allocated for integration and testing
Knowledge Distribution	Key design decisions concentrated with senior members; risk of knowledge silos	Pair programming sessions; architecture documentation shared team-wide; design decisions explained during code reviews
Resource Constraints	Limited access to production-like environments for comprehensive testing	Used Docker Compose to replicate production environment locally; staging environment created on cloud platform

Table 8.2: Administrative Challenges and Solutions

8.3 Lessons Learned

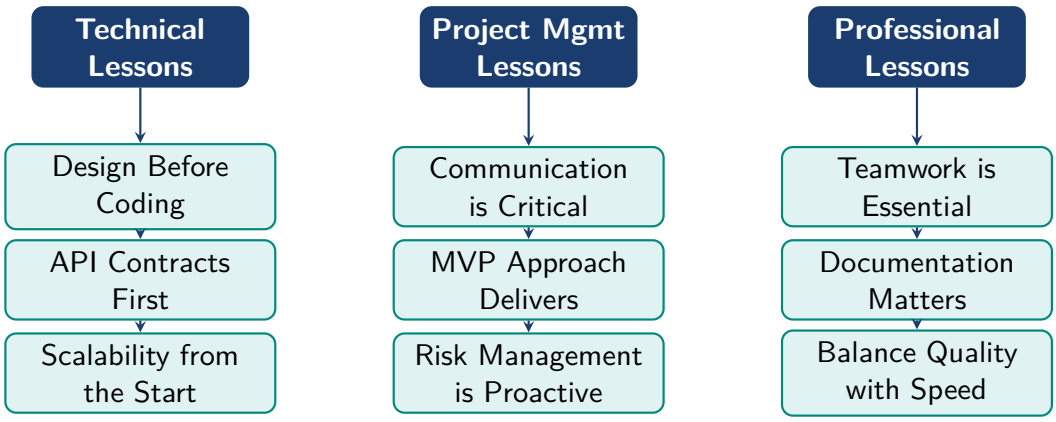

Figure 8.1: Lessons Learned Across Technical, Project Management, and Professional Dimensions

Chapter 9 Contribution to the Organization

9.1 Specific Contributions and Their Impact

9.1.1 Backend Module Development

Module Developed	APIs Created	Key Impact
User Management System	3 endpoints	Role-based access control; 95% test coverage
Restaurant CRUD Operations	8 endpoints	Complete restaurant administration capability
Menu Management Module	5 endpoints	Dynamic menu creation and real-time updates
Inventory Management	4 endpoints	Real-time stock tracking with low-stock alerts
Payment Integration	4 endpoints	Secure multi-gateway payment processing
Analytics Endpoints	6 endpoints	Data aggregation and business intelligence reporting
Coupon System	4 endpoints	Promotional campaign management capability

Table 9.1: Backend Modules Developed and Their Impact

9.1.2 Team Coordination Role

As backend lead and team coordinator, the author served as the primary communication bridge between the frontend and backend development teams. This role involved:

- Defining and maintaining API contracts agreed upon before development began.

- Resolving integration conflicts between frontend and backend implementations.
- Facilitating daily stand-up summaries and distributing tasks based on individual strengths.
- Mentoring junior backend developers on database optimization and API design principles.

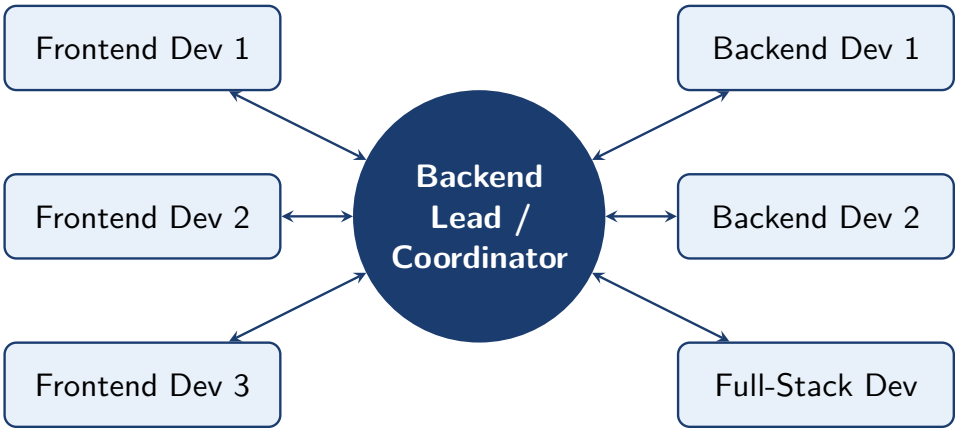

Figure 9.1: Team Coordination Structure During the Attachment

9.1.3 Improvements Suggested and Implemented

Improvement	Implementation	Result
API Documentation Automation	Integrated Swagger/OpenAPI with Express.js for automatic documentation generation	70% reduction in documentation maintenance overhead
Database Performance	Integrated Redis caching for frequently accessed data including menus and user sessions	Reduced database load; improved API response times significantly
Testing Infrastructure	Set up Jest for unit testing and Supertest for API integration testing across all modules	Test coverage reached 85%; early bug detection substantially improved
Error Handling & Logging	Integrated Winston for structured application logging; consistent error handling patterns across all endpoints	Significantly reduced debugging time; improved production visibility

Table 9.2: Improvements Suggested and Implemented During the Attachment

9.2 Feedback from Supervisors

Supervisor Feedback Summary:

- *"Mahfuz demonstrated solid backend development skills and quickly grasped system requirements."* — Strong Technical Execution
- *"Effective in coordinating between teams and ensuring alignment on API contracts."* — Good Coordination Skills
- *"Maintained high code quality standards despite significant time pressure."* — Quality Consciousness
- *"Comprehensive documentation prepared; sets a good example for the entire team."* — Documentation Excellence
- *"Showed initiative in problem-solving and in guiding junior developers."* — Leadership Potential
- *"Could improve in delegating tasks and allowing others to take complete ownership of modules."* — Area for Growth

Overall Assessment: Supervisors confirmed that contributions made by the author significantly accelerated the project timeline and measurably improved overall project quality and code maintainability.

Chapter 10 Conclusion and Recommendations

10.1 Summary of Work Done

Accomplishment Area	Key Metrics
Backend Modules Developed	7 major modules; approximately 3000+ lines of structured, documented code
API Endpoints Created	30+ RESTful API endpoints, fully documented with Swagger
Database Entities Designed	12+ entities with normalized schema in Third Normal Form (3NF)
Test Coverage Achieved	85% through comprehensive unit and integration testing
API Documentation	Complete Swagger/OpenAPI documentation auto-generated from code
CI/CD Pipeline	Configured for staging deployment and automated test execution

Table 10.1: Summary of Technical Accomplishments

10.1.1 Project Outcome

The neoRMS system successfully demonstrated:

- Complete order management and table reservation workflow from placement to fulfillment.
- Real-time synchronization across multiple user interfaces via WebSocket connections.
- Secure payment processing with multi-gateway integration supporting multiple payment methods.

- AI-based analytics providing actionable business insights from operational data.
- Scalable architecture capable of supporting multiple restaurant locations with data isolation.

10.2 Recommendations for Organization

Recommendation	Rationale
Formalize Internship Program	Establish structured learning objectives and mentorship guidelines to maximize intern contribution and learning outcomes
Documentation Templates	Standardized templates for APIs, architecture, and code will maintain consistency across all projects and teams
Knowledge Repository	Capture project learnings and architectural decisions centrally to accelerate onboarding of new team members
CI/CD for All Projects	Comprehensive automated pipelines will catch integration issues early and maintain code quality organization-wide
Formal Code Review Process	Clear review criteria and learning-oriented feedback practices improve code quality and team skill development
Structured Mentorship	Assign experienced engineers as dedicated mentors to junior developers with clear guidelines and regular check-ins

Table 10.2: Recommendations for Brain Station 23

10.3 Recommendations for University

Recommendation	Rationale
Industry-Relevant Curriculum	Include Agile methodologies, modern frameworks, and real-world project management in academic courses
Practical Team Projects	More hands-on projects simulating real-world team-based development scenarios with defined roles
Industry Partnerships	Strengthen partnerships with companies like BS-23 for structured, extended attachment programs
Professional Development	Include engineering ethics, technical documentation, and professional communication alongside technical courses
Industry-Standard Tools	Teach Git, Docker, and CI/CD pipelines in foundational software engineering courses

Table 10.3: Recommendations for the Academic Institution

10.4 Recommendations for Future Interns

- Arrive with foundational knowledge of the technology stack; even basic familiarity significantly reduces the learning curve.
- Be proactive in asking questions and seeking feedback rather than waiting for guidance.
- Document your work consistently throughout the attachment — not only at the end.
- Treat code reviews as learning opportunities rather than evaluations.
- Engage with the full team, not only those in your immediate role, to gain broader perspective.

10.5 Future Research and Development Directions

- **Advanced AI Integration:** Machine learning models for demand forecasting, dynamic pricing, and personalized menu recommendations.
- **IoT Integration:** Connect IoT devices for automated inventory tracking and smart kitchen management.
- **Mobile Application:** Native mobile applications for customers and staff to enhance user experience and accessibility.
- **Multi-tenant Architecture:** Support multiple independent restaurant chains with complete data isolation and per-tenant customization.
- **Advanced Analytics:** Interactive business intelligence dashboards with real-time visualizations and predictive reporting.
- **Blockchain Integration:** Transparent and immutable transaction records for supply chain verification and payment audit trails.

10.6 Final Reflections

Summary

This industrial attachment at Brain Station 23 has been a genuinely transformative experience in the engineering journey of the author. Beyond the technical skills acquired, deep insights were gained into professional software development workflows, team dynamics, and the critical importance of continuous learning in an ever-evolving technology landscape.

The experience reinforced that engineering is not merely about technology — it is fundamentally about understanding people, their real needs, and developing solutions that create meaningful, positive impact. The collaborative culture at BS-23 demonstrated that great software emerges from effective teamwork, clear communication, mutual respect, and a shared commitment to quality and professionalism.

The lessons learned, technical skills developed, professional relationships built, and perspectives gained during these 10 days serve as a strong foundation for continued growth and sustained excellence in software engineering practice.

Bibliography

[1] IEEE, "IEEE Code of Ethics," 2020. [Online]. Available: `https://www.ieee.org/about/corporate/ethics`. [Accessed: March 2026].

[2] R. Martin, *Clean Code: A Handbook of Agile Software Craftsmanship*, Prentice Hall, 2008.

[3] I. Sommerville, *Software Engineering*, 10th ed., Pearson, 2015.

[4] R. Pressman, *Software Engineering: A Practitioner's Approach*, 8th ed., McGraw-Hill, 2014.

[5] K. Beck, *Extreme Programming Explained: Embrace Change*, Addison-Wesley, 2000.

[6] S. Newman, *Building Microservices*, O'Reilly Media, 2015.

[7] M. Fowler, *Patterns of Enterprise Application Architecture*, Addison-Wesley, 2002.

[8] C. J. Date, *An Introduction to Database Systems*, 8th ed., Addison-Wesley, 2003.

[9] S. McConnell, *Code Complete*, 2nd ed., Microsoft Press, 2004.

[10] Agile Alliance, "Manifesto for Agile Software Development," 2001. [Online]. Available: `https://agilemanifesto.org/`. [Accessed: March 2026].

[11] W3C, "Web Content Accessibility Guidelines (WCAG) 2.1," 2023. [Online]. Available: `https://www.w3.org/WAI/WCAG21/quickref/`. [Accessed: March 2026].

[12] OWASP, "OWASP Top 10," 2022. [Online]. Available: `https://owasp.org/www-project-top-ten/`. [Accessed: March 2026].

Chapter A Daily Log Sheet

Day	Date	Key Activities	Deliverables
Day 1	23 Feb 2026	1. Onboarding Meeting 2. GitHub Setup 3. Database 4. SRS 5. BRD 6. Project Planning 7. Scrum	Onboarding complete, GitHub repository initialized, initial DB schema, SRS & BRD drafts, project plan.
Day 2	24 Feb 2026	1. Staff Account Management APIs 2. Learning Docker 3. Learning Prisma 4. Combining backend technologies in full project 5. Restaurant CRUD	Staff account APIs, Docker and Prisma setup, Restaurant CRUD endpoints.
Day 3	25 Feb 2026	1. Menu CRUD 2. Inventory Management 3. Threshold Management	Menu CRUD endpoints, Inventory module, Threshold management module.
Day 4	26 Feb 2026	1. Analytics Endpoints 2. Feature Diagram	Analytics API endpoints, Feature diagram draft.

Continued on next page

Day	Date	Key Activities	Deliverables
Day 5	27 Feb 2026	1. Feature Diagram 2. Aligning code with new schema 3. Coupon and Discount	Finalized feature diagram, schema-aligned codebase, Coupon & Discount module.
Day 6	2 Mar 2026	1. Order code review, debug and merge 2. Table reservation functionality design 3. Promotions and Discounts design	Merged order module, Table reservation design, Promotions and Discounts design.
Day 7	3 Mar 2026	1. Coupon functionality development 2. Low Stock Alert	Coupon functionality and low stock alert feature.
Day 8	4 Mar 2026	1. Payment Integration (SSLCommerz) 2. Collaborating with frontend team	SSLCommerz payment integration and frontend collaboration progress.
Day 9	5 Mar 2026	1. Order Integration (Frontend) 2. Table Reservation (Frontend) 3. Review Integration (Frontend)	Order, table reservation and review modules integrated with frontend.
Day 10	6 Mar 2026	1. Project Presentation	Final project presentation delivered.

Full Daily Log Sheet: `https://bit.ly/4anso9p`

Chapter B Code Snippets

B.1 User Authentication Middleware

The following illustrates the JWT authentication middleware and role-based access control implemented during the project:

Listing B.1: JWT Authentication and RBAC Middleware

```
// JWT Authentication Middleware
const verifyToken = (req, res, next) => {
  const token = req.headers['authorization']?.split(' ')[1];

  if (!token) {
    return res.status(401).json({ error: 'No token provided' });
  }

  try {
    const decoded = jwt.verify(token, process.env.JWT_SECRET);
    req.user = decoded;
    next();
  } catch (error) {
    res.status(401).json({ error: 'Invalid token' });
  }
};

// Role-based Access Control Middleware
const authorize = (roles) => {
  return (req, res, next) => {
    if (!roles.includes(req.user.role)) {
      return res.status(403).json({ error: 'Forbidden' });
    }
    next();
  };
```

```
};
```

B.2 Database Schema

Listing B.2: Core Database Schema — Users, Restaurants, Menu Items

```sql
-- Users table
CREATE TABLE users (
  id          SERIAL PRIMARY KEY,
  email       VARCHAR(255) UNIQUE NOT NULL,
  password    VARCHAR(255) NOT NULL,
  name        VARCHAR(255) NOT NULL,
  role        VARCHAR(50)  NOT NULL,
  created_at  TIMESTAMP    DEFAULT CURRENT_TIMESTAMP
);

-- Restaurants table
CREATE TABLE restaurants (
  id          SERIAL PRIMARY KEY,
  name        VARCHAR(255) NOT NULL,
  address     VARCHAR(500),
  city        VARCHAR(100),
  owner_id    INTEGER REFERENCES users(id),
  created_at  TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

-- Menu Items table
CREATE TABLE menu_items (
  id            SERIAL PRIMARY KEY,
  restaurant_id INTEGER REFERENCES restaurants(id),
  name          VARCHAR(255)    NOT NULL,
  description   TEXT,
  price         DECIMAL(10, 2),
  available     BOOLEAN         DEFAULT true,
  created_at    TIMESTAMP       DEFAULT CURRENT_TIMESTAMP
);
```

Chapter C Additional Resources

C.1 GitHub Repository

Source code for the neoRMS project is available at:

> `https://github.com/naim1405/neoRMS`
>
> The repository contains complete source code (backend and frontend), database migration scripts, Swagger API documentation, a README with full setup instructions, and contributing guidelines for future developers.

C.2 Project Documentation

Complete project documentation including:

- Architecture design document
- API specification and Swagger/OpenAPI documentation
- Database schema and ER diagram documentation
- Deployment and environment setup guides

C.3 Certificates and Recognition

- Brain Station 23 Industrial Attachment Completion Certificate
- Performance evaluation report from project supervisors
- Letter of recommendation from BS-23 management